

Checkpointing

1. Overview

Checkpointing is a fault-tolerance technique where a system periodically saves its **current progress/state** so that if a machine crashes, processing can resume from the last saved point instead of restarting from the beginning.

It is heavily used in:

- Batch processing systems
- ETL pipelines
- Streaming systems
- Data aggregation jobs
- Distributed computing
- Kafka consumers
- Apache Flink / Spark Streaming
- Long-running background jobs

2. Why Checkpointing is Needed

Imagine a machine processing **10 million records**.

Without checkpointing:

None

Processed 9 million records

Machine crashes

Restart from 0

Huge waste of time and compute.

With checkpointing:

None

Checkpoint saved at 9 million

Crash occurs

Restart from 9 million + 1

This saves time, cost, and ensures reliability.

3. Core Idea

After processing a chunk of work:

1. Process subset of data
2. Save result/output
3. Save progress marker (checkpoint)

If failure occurs:

- New machine starts
- Reads latest checkpoint
- Continues from saved state

4. Simple Example

Suppose system processes records in batches of 1000.

None

Processed records 1-1000 → checkpoint = 1000

Processed records 1001-2000 → checkpoint = 2000

Processed records 2001-3000 → crash

Recovery:

None

Restart from checkpoint = 2000

Resume from 2001

5. Real-World ETL Example

Pipeline:

None

Database → Transform → Data Warehouse

Process:

- Read 10,000 rows
- Clean / transform data
- Insert into warehouse
- Save checkpoint = last processed row ID

If worker crashes:

- Resume from last row ID

6. Streaming Systems Example

Streaming systems run continuously.

Example:

None

User Click Events

Payment Events

Sensor Data

Logs

Need checkpointing because system may run forever.

Instead of restarting full stream, resume from saved state.

7. Kafka + Checkpointing

Kafka uses Offsets

Every message in a Kafka partition has an **offset**.

Example:

None

Partition-0:

offset 0

offset 1

offset 2

offset 3

...

Consumer reads messages and commits checkpoint using offset.

If crash occurs:

None

Last committed offset = 5000

Restart consumer

Resume from offset 5001

This is checkpointing via offsets.

8. Apache Flink Checkpointing

Apache Flink supports advanced distributed checkpointing.

It stores:

- Operator state
- Window state
- Kafka offsets
- Timers
- Intermediate computation state

If node fails:

- Restore entire distributed pipeline from checkpoint

Very useful for exactly-once processing.

9. Checkpoint Contains What?

Depends on system.

Possible contents:

- Last processed record ID
- Kafka offset
- Aggregation counters
- Running totals
- Hash maps
- Window state
- User session state
- File position
- Progress timestamp

10. Where Checkpoints Are Stored

Common storage options:

Durable Storage

- S3
- HDFS
- GCS
- Distributed DB
- Persistent disk

Fast Local Storage

- SSD
- Local disk (less reliable)

Best practice: durable replicated storage.

11. Checkpoint Frequency Trade-off

Frequent Checkpoints

Pros:

- Less work lost on failure

Cons:

- Higher overhead

Rare Checkpoints

Pros:

- Better runtime performance

Cons:

- More reprocessing after crash

Example:

None

Checkpoint every 5 sec → safer

Checkpoint every 1 hour → risky

12. Exactly Once vs At Least Once

Checkpointing helps processing guarantees.

At Least Once

After restart, some records may reprocess.

Duplicate possible.

Exactly Once

Restore state + offsets consistently.

No duplicates, no loss.

Used by Flink/Kafka transactions.

13. Checkpoint Flow Diagram

None

Read Data Batch



Process Batch



Write Output



Save Checkpoint



Next Batch

Crash Recovery:

None

Crash



New Worker Starts



Read Last Checkpoint



Resume Processing

14. Aggregation Example

Suppose calculating sum of billions of values.

Without checkpoint:

None

Crash at 90%

Need full recomputation

With checkpoint:

None

Saved:

processed_count = 900M

sum = 8.2B

Resume from there

15. Important Design Considerations

A. Atomicity

Need:

None

Output write succeeds

AND

Checkpoint write succeeds

Otherwise inconsistency occurs.

B. Idempotency

If replay happens, processing should be safe.

C. State Size

Large checkpoints are expensive.

Need incremental checkpoints.

D. Durability

Checkpoint must survive machine failure.

E. Recovery Speed

Restore should be fast.

16. Incremental Checkpointing

Instead of storing full state every time:

None

Checkpoint1 = full state

Checkpoint2 = changes only

Checkpoint3 = changes only

Used in Flink / RocksDB backends.

Saves storage + time.

17. Interview Answer

Q: How do streaming systems recover from crashes?

Answer:

They periodically checkpoint processing state and progress offsets. If a worker crashes, another worker restores from the latest checkpoint and resumes processing. Systems like Kafka use offsets, while Flink checkpoints full distributed operator state.

18. Real Systems Using Checkpointing

- Apache Kafka Consumers
- Apache Flink
- Spark Structured Streaming
- Hadoop jobs
- Airflow ETL jobs
- Database replication logs
- ML training jobs

19. Best Practices

- Use durable storage
- Keep checkpoint intervals balanced
- Make writes idempotent
- Monitor checkpoint failures
- Use incremental checkpoints

- Periodically test recovery
- Replicate checkpoints

20. Short Revision Notes

None

Checkpointing = Save progress periodically

Crash occurs:

→ Resume from saved checkpoint

Used in:

Kafka, Flink, ETL, Streaming

Stores:

Offsets, state, counters, progress

Tradeoff:

Frequent = safe but overhead

Rare = faster but more replay